

Zoho Coding Round — DSA Prep (C++)

Ordered by Priority (based on typical Zoho hiring test patterns)

Zoho's coding rounds lean heavily on **arrays, strings, matrices, recursion/backtracking, stacks, and linked lists**, with a lighter but real presence of **DP and graphs**. The problems below are grouped into 4 priority tiers. Practice Tier 1 until it's automatic, then move down.

TIER 1 — Must Know Cold (asked most frequently)

1. Valid Parentheses — LeetCode 20

Problem: Given a string containing `()`, `{}`, `[]`, determine if the brackets are balanced.

Concept: Stack, matching pairs.

```
cpp

#include <bits/stdc++.h>
using namespace std;

bool isValid(string s) {
    stack<char> st;
    unordered_map<char, char> match = {{'}', '{'}, {'}', '('}, {'}', '['}, {'}', '{'}};

    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
        } else {
            if (st.empty() || st.top() != match[c]) return false;
            st.pop();
        }
    }
    return st.empty();
}

int main() {
    cout << (isValid("{[()]}" ) ? "true" : "false") << endl; // true
}
```

Complexity: $O(n)$ time, $O(n)$ space.

2. Two Sum — LeetCode 1

Problem: Return indices of the two numbers that add up to target. **Concept:** HashMap.

cpp

```
#include <bits/stdc++.h>
using namespace std;

vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int,int> seen; // value -> index
    for (int i = 0; i < (int)nums.size(); i++) {
        int need = target - nums[i];
        if (seen.count(need)) return {seen[need], i};
        seen[nums[i]] = i;
    }
    return {};
}

int main() {
    vector<int> nums = {2,7,11,15};
    auto res = twoSum(nums, 9);
    cout << res[0] << " " << res[1] << endl; // 0 1
}
```

Complexity: $O(n)$ time, $O(n)$ space.

3. Reverse Linked List — LeetCode 206

Problem: Reverse a singly linked list. **Concept:** Pointer manipulation.

cpp

```

#include <bits/stdc++.h>
using namespace std;

struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* curr = head;
    while (curr) {
        ListNode* nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }
    return prev;
}

void printList(ListNode* head) {
    while (head) { cout << head->val << " "; head = head->next; }
    cout << endl;
}

int main() {
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(4);
    ListNode* rev = reverseList(head);
    printList(rev); // 4 3 2 1
}

```

Complexity: $O(n)$ time, $O(1)$ space.

4. Rotate Matrix 90 Degrees (Zoho favorite)

Problem: Rotate an $N \times N$ matrix by 90 degrees clockwise, in place.

cpp

```

#include <bits/stdc++.h>
using namespace std;

void rotate(vector<vector<int>>& matrix) {
    int n = matrix.size();
    // Step 1: transpose
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            swap(matrix[i][j], matrix[j][i]);

    // Step 2: reverse each row
    for (int i = 0; i < n; i++)
        reverse(matrix[i].begin(), matrix[i].end());
}

int main() {
    vector<vector<int>> m = {{1,2,3},{4,5,6},{7,8,9}};
    rotate(m);
    for (auto& row : m) {
        for (int x : row) cout << x << " ";
        cout << endl;
    }
    // 7 4 1
    // 8 5 2
    // 9 6 3
}

```

Complexity: $O(n^2)$ time, $O(1)$ extra space.

5. Spiral Matrix — LeetCode 54

Problem: Print/return matrix elements in spiral order.

cpp

```

#include <bits/stdc++.h>
using namespace std;

vector<int> spiralOrder(vector<vector<int>>& matrix) {
    vector<int> res;
    if (matrix.empty()) return res;
    int top = 0, bottom = matrix.size() - 1;
    int left = 0, right = matrix[0].size() - 1;

    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) res.push_back(matrix[top][j]);
        top++;
        for (int i = top; i <= bottom; i++) res.push_back(matrix[i][right]);
        right--;
        if (top <= bottom) {
            for (int j = right; j >= left; j--) res.push_back(matrix[bottom][j]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) res.push_back(matrix[i][left]);
            left++;
        }
    }
    return res;
}

int main() {
    vector<vector<int>> m = {{1,2,3},{4,5,6},{7,8,9}};
    for (int x : spiralOrder(m)) cout << x << " ";
    // 1 2 3 6 9 8 7 4 5
}

```

Complexity: $O(n*m)$ time, $O(1)$ extra space (excluding output).

6. Print All Permutations

Problem: Print all permutations of a string, e.g. "ABC".

cpp

```

#include <bits/stdc++.h>
using namespace std;

void permute(string& s, int l) {
    if (l == (int)s.size()) {
        cout << s << endl;
        return;
    }
    for (int i = l; i < (int)s.size(); i++) {
        swap(s[l], s[i]);
        permute(s, l + 1);
        swap(s[l], s[i]); // backtrack
    }
}

int main() {
    string s = "ABC";
    permute(s, 0);
    // ABC ACB BAC BCA CBA CAB (order may vary based on swap approach;
    // use next_permutation on sorted string for lexicographic order)
}

```

Lexicographic order variant (matches expected output exactly):

```

cpp

#include <bits/stdc++.h>
using namespace std;

int main() {
    string s = "ABC";
    sort(s.begin(), s.end());
    do {
        cout << s << endl;
    } while (next_permutation(s.begin(), s.end()));
    // ABC ACB BAC BCA CAB CBA
}

```

Complexity: $O(n! * n)$ time.

Problem: Check if two strings are anagrams.

cpp

```
#include <bits/stdc++.h>
using namespace std;

bool isAnagram(string s, string t) {
    if (s.size() != t.size()) return false;
    int freq[26] = {0};
    for (char c : s) freq[c - 'a']++;
    for (char c : t) freq[c - 'a']--;
    for (int f : freq) if (f != 0) return false;
    return true;
}

int main() {
    cout << (isAnagram("anagram", "nagaram") ? "true" : "false") << endl; // tr
}
```

Complexity: $O(n)$ time, $O(1)$ space.

8. Longest Substring Without Repeating Characters — LeetCode 3

Problem: Find length of the longest substring without repeating characters. **Concept:** Sliding window.

cpp

```

#include <bits/stdc++.h>
using namespace std;

int lengthOfLongestSubstring(string s) {
    unordered_map<char,int> lastSeen;
    int maxLen = 0, start = 0;
    for (int i = 0; i < (int)s.size(); i++) {
        if (lastSeen.count(s[i]) && lastSeen[s[i]] >= start) {
            start = lastSeen[s[i]] + 1;
        }
        lastSeen[s[i]] = i;
        maxLen = max(maxLen, i - start + 1);
    }
    return maxLen;
}

int main() {
    cout << lengthOfLongestSubstring("abcabcbb") << endl; // 3
}

```

Complexity: $O(n)$ time, $O(\min(n, \text{charset}))$ space.

TIER 2 — High Priority

9. Best Time to Buy and Sell Stock — LeetCode 121

Concept: Greedy, single pass.

```
cpp
```



```
#include <bits/stdc++.h>
using namespace std;

int maxProfit(vector<int>& prices) {
    int minPrice = INT_MAX, maxProfit = 0;
    for (int p : prices) {
        minPrice = min(minPrice, p);
        maxProfit = max(maxProfit, p - minPrice);
    }
    return maxProfit;
}

int main() {
    vector<int> prices = {7,1,5,3,6,4};
    cout << maxProfit(prices) << endl; // 5
}
```

Complexity: $O(n)$ time, $O(1)$ space.

10. Product of Array Except Self — LeetCode 238

Concept: Prefix/suffix product, no division.

cpp

```

#include <bits/stdc++.h>
using namespace std;

vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> res(n, 1);
    int prefix = 1;
    for (int i = 0; i < n; i++) {
        res[i] = prefix;
        prefix *= nums[i];
    }
    int suffix = 1;
    for (int i = n - 1; i >= 0; i--) {
        res[i] *= suffix;
        suffix *= nums[i];
    }
    return res;
}

int main() {
    vector<int> nums = {1,2,3,4};
    for (int x : productExceptSelf(nums)) cout << x << " ";
    // 24 12 8 6
}

```

Complexity: $O(n)$ time, $O(1)$ extra space (excluding output array).

11. Container With Most Water — LeetCode 11

Concept: Two pointers.

cpp

```
#include <bits/stdc++.h>
using namespace std;

int maxArea(vector<int>& height) {
    int l = 0, r = height.size() - 1, best = 0;
    while (l < r) {
        int area = min(height[l], height[r]) * (r - l);
        best = max(best, area);
        if (height[l] < height[r]) l++;
        else r--;
    }
    return best;
}

int main() {
    vector<int> h = {1,8,6,2,5,4,8,3,7};
    cout << maxArea(h) << endl; // 49
}
```

Complexity: $O(n)$ time, $O(1)$ space.

12. Merge Two Sorted Lists — LeetCode 21

cpp

```

#include <bits/stdc++.h>
using namespace std;

struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* mergeTwoLists(ListNode* l1, ListNode* l2) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (l1 && l2) {
        if (l1->val <= l2->val) { tail->next = l1; l1 = l1->next; }
        else { tail->next = l2; l2 = l2->next; }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}

void printList(ListNode* head) {
    while (head) { cout << head->val << " "; head = head->next; }
    cout << endl;
}

int main() {
    ListNode* l1 = new ListNode(1);
    l1->next = new ListNode(3);
    l1->next->next = new ListNode(5);

    ListNode* l2 = new ListNode(2);
    l2->next = new ListNode(4);
    l2->next->next = new ListNode(6);

    printList(mergeTwoLists(l1, l2)); // 1 2 3 4 5 6
}

```

Complexity: $O(n + m)$ time, $O(1)$ extra space.

Concept: Floyd's cycle detection (slow/fast pointers).

```
cpp

#include <bits/stdc++.h>
using namespace std;

struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

bool hasCycle(ListNode *head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

int main() {
    ListNode* a = new ListNode(1);
    ListNode* b = new ListNode(2);
    a->next = b;
    b->next = a; // cycle
    cout << (hasCycle(a) ? "true" : "false") << endl; // true
}
```

Complexity: O(n) time, O(1) space.

14. String Compression — LeetCode 443

Concept: Two pointers / run-length encoding.

```
cpp
```

```

#include <bits/stdc++.h>
using namespace std;

string compress(vector<char>& chars) {
    string result;
    int i = 0, n = chars.size();
    while (i < n) {
        char c = chars[i];
        int count = 0;
        while (i < n && chars[i] == c) { i++; count++; }
        result += c;
        if (count > 1) result += to_string(count);
    }
    return result;
}

int main() {
    vector<char> chars = {'a', 'a', 'b', 'b', 'c', 'c', 'c'};
    cout << compress(chars) << endl; // a2b2c3
}

```

Complexity: $O(n)$ time, $O(1)$ extra space (in-place variant possible).

15. Longest Palindromic Substring — LeetCode 5

Concept: Expand around center.

cpp

```

#include <bits/stdc++.h>
using namespace std;

string expand(const string& s, int l, int r) {
    while (l >= 0 && r < (int)s.size() && s[l] == s[r]) { l--; r++; }
    return s.substr(l + 1, r - l - 1);
}

string longestPalindrome(string s) {
    string best = "";
    for (int i = 0; i < (int)s.size(); i++) {
        string odd = expand(s, i, i);
        string even = expand(s, i, i + 1);
        if (odd.size() > best.size()) best = odd;
        if (even.size() > best.size()) best = even;
    }
    return best;
}

int main() {
    cout << longestPalindrome("babad") << endl; // "bab" (or "aba")
}

```

Complexity: $O(n^2)$ time, $O(1)$ extra space.

TIER 3 — Medium Priority (DP / Graph basics)

16. Climbing Stairs — LeetCode 70

Concept: Fibonacci-style DP.

```
cpp
```

```
#include <bits/stdc++.h>
using namespace std;

int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    return b;
}

int main() {
    cout << climbStairs(5) << endl; // 8
}
```

Complexity: $O(n)$ time, $O(1)$ space.

17. Coin Change — LeetCode 322

Concept: Bottom-up DP.

cpp


```

#include <bits/stdc++.h>
using namespace std;

int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int c : coins) {
            if (c <= i && dp[i - c] != INT_MAX)
                dp[i] = min(dp[i], dp[i - c] + 1);
        }
    }
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}

int main() {
    vector<int> coins = {1,2,5};
    cout << coinChange(coins, 11) << endl; // 3 (5+5+1)
}

```

Complexity: $O(\text{amount} * \text{coins.size}())$ time, $O(\text{amount})$ space.

18. House Robber — LeetCode 198 / House Robber II — LeetCode 213

Concept: DP; II handles a circular arrangement by running the linear version twice (excluding first, excluding last).

cpp

```

#include <bits/stdc++.h>
using namespace std;

// House Robber I (linear row of houses)
int robLinear(vector<int>& nums) {
    int prev = 0, curr = 0;
    for (int n : nums) {
        int temp = max(curr, prev + n);
        prev = curr;
        curr = temp;
    }
    return curr;
}

// House Robber II (circular arrangement)
int rob(vector<int>& nums) {
    int n = nums.size();
    if (n == 1) return nums[0];
    vector<int> excludeLast(nums.begin(), nums.end() - 1);
    vector<int> excludeFirst(nums.begin() + 1, nums.end());
    return max(robLinear(excludeLast), robLinear(excludeFirst));
}

int main() {
    vector<int> nums = {2,3,2};
    cout << rob(nums) << endl; // 3
}

```

Complexity: $O(n)$ time, $O(1)$ extra space (excluding the two sub-vectors).

19. Number of Islands — LeetCode 200

Concept: DFS/BFS grid traversal.

cpp

```

#include <bits/stdc++.h>
using namespace std;

void dfs(vector<vector<char>>& grid, int i, int j) {
    int rows = grid.size(), cols = grid[0].size();
    if (i < 0 || j < 0 || i >= rows || j >= cols || grid[i][j] != '1') return;
    grid[i][j] = '0'; // mark visited
    dfs(grid, i+1, j);
    dfs(grid, i-1, j);
    dfs(grid, i, j+1);
    dfs(grid, i, j-1);
}

int numIslands(vector<vector<char>>& grid) {
    int count = 0;
    for (int i = 0; i < (int)grid.size(); i++)
        for (int j = 0; j < (int)grid[0].size(); j++)
            if (grid[i][j] == '1') { count++; dfs(grid, i, j); }
    return count;
}

int main() {
    vector<vector<char>> grid = {
        {'1', '1', '0'},
        {'1', '0', '0'},
        {'0', '0', '1'}
    };
    cout << numIslands(grid) << endl; // 2
}

```

Complexity: $O(\text{rowscols})$ time, $O(\text{rowscols})$ recursion stack worst case.

20. Flood Fill — LeetCode 733

Concept: DFS.

cpp

```

#include <bits/stdc++.h>
using namespace std;

void fill(vector<vector<int>>& image, int i, int j, int oldColor, int newColor)
{
    int rows = image.size(), cols = image[0].size();
    if (i < 0 || j < 0 || i >= rows || j >= cols) return;
    if (image[i][j] != oldColor || image[i][j] == newColor) return;
    image[i][j] = newColor;
    fill(image, i+1, j, oldColor, newColor);
    fill(image, i-1, j, oldColor, newColor);
    fill(image, i, j+1, oldColor, newColor);
    fill(image, i, j-1, oldColor, newColor);
}

vector<vector<int>> floodFill(vector<vector<int>>& image, int sr, int sc, int color)
{
    int oldColor = image[sr][sc];
    if (oldColor != color) fill(image, sr, sc, oldColor, color);
    return image;
}

int main() {
    vector<vector<int>> image = {{1,1,1},{1,1,0},{1,0,1}};
    auto res = floodFill(image, 1, 1, 2);
    for (auto& row : res) {
        for (int x : row) cout << x << " ";
        cout << endl;
    }
    // 2 2 2
    // 2 2 0
    // 2 0 1
}

```

Complexity: $O(\text{rows} \times \text{cols})$ time.

21. Rotting Oranges — LeetCode 994

Concept: Multi-source BFS.

cpp

```

#include <bits/stdc++.h>
using namespace std;

int orangesRotting(vector<vector<int>>& grid) {
    int rows = grid.size(), cols = grid[0].size();
    queue<pair<int,int>> q;
    int fresh = 0;

    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++) {
            if (grid[i][j] == 2) q.push({i, j});
            if (grid[i][j] == 1) fresh++;
        }

    if (fresh == 0) return 0;

    int minutes = 0;
    vector<pair<int,int>> dirs = {{1,0},{-1,0},{0,1},{0,-1}};

    while (!q.empty()) {
        int size = q.size();
        bool rottedAny = false;
        for (int k = 0; k < size; k++) {
            auto [i, j] = q.front(); q.pop();
            for (auto& d : dirs) {
                int ni = i + d.first, nj = j + d.second;
                if (ni >= 0 && nj >= 0 && ni < rows && nj < cols && grid[ni][nj] == 1) {
                    grid[ni][nj] = 2;
                    fresh--;
                    q.push({ni, nj});
                    rottedAny = true;
                }
            }
        }
        if (rottedAny) minutes++;
    }

    return fresh == 0 ? minutes : -1;
}

int main() {
    vector<vector<int>> grid = {{2,1,1},{1,1,0},{0,1,1}};

```

```
    cout << orangesRotting(grid) << endl; // 4
}
```

Complexity: $O(\text{rows} \times \text{cols})$ time and space.

TIER 4 — Good to Know (asked less often, but seen in advanced rounds)

22. Next Greater Element I — LeetCode 496

Concept: Monotonic stack.

```
cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> nextGreaterElement(vector<int>& nums1, vector<int>& nums2) {
    unordered_map<int,int> nextGreater;
    stack<int> st;
    for (int n : nums2) {
        while (!st.empty() && st.top() < n) {
            nextGreater[st.top()] = n;
            st.pop();
        }
        st.push(n);
    }
    vector<int> res;
    for (int n : nums1) res.push_back(nextGreater.count(n) ? nextGreater[n] : -1);
    return res;
}

int main() {
    vector<int> nums1 = {4,1,2}, nums2 = {4,1,2};
    for (int x : nextGreaterElement(nums1, nums2)) cout << x << " ";
    // -1 2 -1
}
```

Complexity: $O(n + m)$ time, $O(n)$ space.

23. Daily Temperatures — LeetCode 739

Concept: Monotonic stack (indices).

```
cpp

#include <bits/stdc++.h>
using namespace std;

vector<int> dailyTemperatures(vector<int>& temps) {
    int n = temps.size();
    vector<int> res(n, 0);
    stack<int> st; // stores indices
    for (int i = 0; i < n; i++) {
        while (!st.empty() && temps[st.top()] < temps[i]) {
            int idx = st.top(); st.pop();
            res[idx] = i - idx;
        }
        st.push(i);
    }
    return res;
}

int main() {
    vector<int> temps = {73,74,75,71,69,72,76,73};
    for (int x : dailyTemperatures(temps)) cout << x << " ";
    // 1 1 4 2 1 1 0 0
}
```

Complexity: $O(n)$ time, $O(n)$ space.

24. Simplify Path — LeetCode 71

Concept: Stack + string parsing.

```
cpp
```

```

#include <bits/stdc++.h>
using namespace std;

string simplifyPath(string path) {
    vector<string> stack;
    stringstream ss(path);
    string token;

    while (getline(ss, token, '/')) {
        if (token == "" || token == ".") continue;
        if (token == "..") {
            if (!stack.empty()) stack.pop_back();
        } else {
            stack.push_back(token);
        }
    }

    string result;
    for (auto& dir : stack) result += "/" + dir;
    return result.empty() ? "/" : result;
}

int main() {
    cout << simplifyPath("/home//foo/") << endl; // /home/foo
}

```

Complexity: $O(n)$ time, $O(n)$ space.

25. Longest Increasing Subsequence — LeetCode 300

Concept: DP ($O(n^2)$) or Patience Sorting / Binary Search ($O(n \log n)$).

```
cpp
```



```

#include <bits/stdc++.h>
using namespace std;

// O(n log n) approach
int lengthOfLIS(vector<int>& nums) {
    vector<int> tails; // tails[i] = smallest possible tail of an increasing subsequence of length i+1
    for (int n : nums) {
        auto it = lower_bound(tails.begin(), tails.end(), n);
        if (it == tails.end()) tails.push_back(n);
        else *it = n;
    }
    return tails.size();
}

int main() {
    vector<int> nums = {10,9,2,5,3,7,101,18};
    cout << lengthOfLIS(nums) << endl; // 4 (2,5,7,101 or 2,3,7,18)
}

```

Complexity: $O(n \log n)$ time, $O(n)$ space.

26. Course Schedule — LeetCode 207

Concept: Graph cycle detection via topological sort (Kahn's algorithm).

cpp

```

#include <bits/stdc++.h>
using namespace std;

bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
    vector<vector<int>> graph(numCourses);
    vector<int> indegree(numCourses, 0);

    for (auto& p : prerequisites) {
        graph[p[1]].push_back(p[0]);
        indegree[p[0]]++;
    }

    queue<int> q;
    for (int i = 0; i < numCourses; i++)
        if (indegree[i] == 0) q.push(i);

    int visited = 0;
    while (!q.empty()) {
        int course = q.front(); q.pop();
        visited++;
        for (int next : graph[course]) {
            if (--indegree[next] == 0) q.push(next);
        }
    }

    return visited == numCourses;
}

int main() {
    vector<vector<int>> prereq = {{1,0}};
    cout << (canFinish(2, prereq) ? "true" : "false") << endl; // true
}

```

Complexity: $O(V + E)$ time, $O(V + E)$ space.

27. N-Queens — LeetCode 51

Concept: Backtracking.

cpp

```

#include <bits/stdc++.h>
using namespace std;

void solve(int row, int n, vector<string>& board,
           vector<bool>& cols, vector<bool>& diag1, vector<bool>& diag2,
           vector<vector<string>>& results) {
    if (row == n) {
        results.push_back(board);
        return;
    }
    for (int col = 0; col < n; col++) {
        int d1 = row + col, d2 = row - col + n - 1;
        if (cols[col] || diag1[d1] || diag2[d2]) continue;

        board[row][col] = 'Q';
        cols[col] = diag1[d1] = diag2[d2] = true;

        solve(row + 1, n, board, cols, diag1, diag2, results);

        board[row][col] = '.';
        cols[col] = diag1[d1] = diag2[d2] = false;
    }
}

vector<vector<string>> solveNQueens(int n) {
    vector<vector<string>> results;
    vector<string> board(n, string(n, '.'));
    vector<bool> cols(n, false), diag1(2*n, false), diag2(2*n, false);
    solve(0, n, board, cols, diag1, diag2, results);
    return results;
}

int main() {
    auto results = solveNQueens(4);
    for (auto& sol : results) {
        for (auto& row : sol) cout << row << endl;
        cout << "---" << endl;
    }
    // One valid solution:
    // .Q..
    // ...Q
    // Q...

```

```
// ..Q.  
}
```

Complexity: $O(n!)$ worst case time.

28. LRU Cache — LeetCode 146

Concept: HashMap + Doubly Linked List for $O(1)$ get/put.

```
cpp
```

```

#include <bits/stdc++.h>
using namespace std;

class LRUCache {
    int capacity;
    list<pair<int,int>> order; // front = most recently used, back = least recently used
    unordered_map<int, list<pair<int,int>>::iterator> cache; // key -> iterator

public:
    LRUCache(int capacity) : capacity(capacity) {}

    int get(int key) {
        if (cache.find(key) == cache.end()) return -1;
        auto it = cache[key];
        int val = it->second;
        order.erase(it);
        order.push_front({key, val});
        cache[key] = order.begin();
        return val;
    }

    void put(int key, int value) {
        if (cache.find(key) != cache.end()) {
            order.erase(cache[key]);
        } else if ((int)cache.size() >= capacity) {
            auto last = order.back();
            cache.erase(last.first);
            order.pop_back();
        }
        order.push_front({key, value});
        cache[key] = order.begin();
    }
};

int main() {
    LRUCache cache(2);
    cache.put(1, 1);
    cache.put(2, 2);
    cout << cache.get(1) << endl; // 1
    cache.put(3, 3); // evicts key 2
    cout << cache.get(2) << endl; // -1
}

```

Complexity: $O(1)$ time for get/put, $O(\text{capacity})$ space.

29. Asteroid Collision — LeetCode 735

Problem: Asteroids move right (positive) or left (negative). When two collide, the smaller explodes; equal size, both explode. **Concept:** Stack.

```
cpp

#include <bits/stdc++.h>
using namespace std;

vector<int> asteroidCollision(vector<int>& asteroids) {
    vector<int> st; // acting as a stack

    for (int a : asteroids) {
        bool alive = true;
        while (alive && a < 0 && !st.empty() && st.back() > 0) {
            if (st.back() < -a) {
                st.pop_back(); // top explodes, current continues
                continue;
            } else if (st.back() == -a) {
                st.pop_back(); // both explode
            }
            alive = false; // current explodes
        }
        if (alive) st.push_back(a);
    }
    return st;
}

int main() {
    vector<int> asteroids = {5,10,-5};
    for (int x : asteroidCollision(asteroids)) cout << x << " ";
    // 5 10
}
```

Complexity: $O(n)$ time, $O(n)$ space.

30. Max Area of Island — LeetCode 695

Problem: Find the maximum area of an island (connected group of 1's) in a grid. **Concept:**

DFS/BFS.

cpp

```
#include <bits/stdc++.h>
using namespace std;

int dfs(vector<vector<int>>& grid, int i, int j) {
    int rows = grid.size(), cols = grid[0].size();
    if (i < 0 || j < 0 || i >= rows || j >= cols || grid[i][j] != 1) return 0;
    grid[i][j] = 0; // mark visited
    int area = 1;
    area += dfs(grid, i+1, j);
    area += dfs(grid, i-1, j);
    area += dfs(grid, i, j+1);
    area += dfs(grid, i, j-1);
    return area;
}

int maxAreaOfIsland(vector<vector<int>>& grid) {
    int best = 0;
    for (int i = 0; i < (int)grid.size(); i++)
        for (int j = 0; j < (int)grid[0].size(); j++)
            if (grid[i][j] == 1)
                best = max(best, dfs(grid, i, j));
    return best;
}

int main() {
    vector<vector<int>> grid = {
        {0,0,1,0},
        {0,1,1,0},
        {0,0,0,0}
    };
    cout << maxAreaOfIsland(grid) << endl; // 3
}
```

Complexity: $O(\text{rows} \times \text{cols})$ time.

Quick-Reference Priority Summary

Tier	Focus Area	Problems
1	Core patterns Zoho tests constantly	Valid Parentheses, Two Sum, Reverse Linked List, Rotate Matrix, Spiral Matrix, Permutations, Valid Anagram, Longest Substring w/o Repeat
2	Common follow-ups / two-pointer & lists	Buy/Sell Stock, Product Except Self, Container With Most Water, Merge Two Lists, Linked List Cycle, String Compression, Longest Palindromic Substring
3	DP & Graph basics	Climbing Stairs, Coin Change, House Robber I/II, Number of Islands, Flood Fill, Rotting Oranges
4	Advanced / less frequent but seen	Next Greater Element, Daily Temperatures, Simplify Path, LIS, Course Schedule, N-Queens, LRU Cache, Asteroid Collision, Max Area of Island

Study tip: In a Zoho test, matrix and recursion/backtracking questions (rotate, spiral, permutations, N-Queens) tend to appear as the "logic" round, while stack/array/string problems appear as warm-up questions. LRU Cache and Course Schedule are more common in the technical interview round than the initial coding test.